

Sciex ZT Scanning-DIA: Batched / Streaming Meta-Scan Collapse

Backlog item 5b — design + implementation plan

Pioneer.jl branch `feat/sciex-zt-scanning`

2026-07-20

Abstract

The peak-memory driver of a ZT search is the full raw per-(precursor, scan) post-deconvolution PSM table (33–40M rows), which is materialized in its entirety and *then* reduced $\sim 11\times$ by `collapse_to_metascans`. Because the collapse is *local* (each meta-PSM needs only $\pm k$ neighbor scans of one precursor), it can run *per contiguous-scan batch*, *before* the concatenation that builds the giant table — concatenating ~ 3 M meta-PSMs instead of ~ 35 M raw rows. This document records the measured local baseline, gives the correctness argument (why the result stays byte-identical), and shows the concrete code changes with examples.

1 Measured baseline (this machine)

Single file (Condition.A.REP1), PIONEER_ZT_METASCAN_K=6, 10 threads, `gcthreads 5,1`, macOS, 64 GB RAM. Wall ≈ 7.0 min.

Metric	Value
Peak memory footprint (macOS time -1, authoritative)	22.30 GB
Pioneer internal “peak RSS” (counts mmap’d arrow)	34.38 GB
Main Search RSS jump	+24.51 GB
<i>Raw post-deconv table</i> (33,185,795 rows $\times$ 55 col)	6.34 GB
<code>permute_by_precursor</code> (global sort churn)	7.45 GB
<code>metascan_collapse</code> (cumulative alloc)	3.68 GB
Meta-PSM table (2,950,447 rows $\times$ 111 col)	1.11 GB
<code>library_search</code> deconv (cumulative alloc)	31.17 GB

Collapse invariant (the fast byte-identity check for 5b, independent of MBR so a single-file run suffices): 33,185,795 $\rightarrow$ 2,950,447 meta-PSMs. Full-search reference: 2-file result 57,030 / 8,568.

5b directly removes the two bold rows: the raw table never fully materializes (it is collapsed per batch to ~ 1.1 GB of meta-PSMs), and the 7.45 GB global `permute_by_precursor` disappears (each batch sorts $\sim 1/\#\text{threads}$ of the data).

2 Current flow

In `LibrarySearch.jl`, per file:

1. `partition_scans` splits scans across threads **round-robin** in batches of 10 (`_distribute_scans_to_threads`) — interleaved for load balance.
2. each thread's `process_scans_fused!` deconvolves its scans $\rightarrow$ a DataFrame of raw per-(precursor,scan) PSMs.
3. `result = vcat(fetched...)` (`LibrarySearch.jl:227`) — *materializes the full 33–40M-row table*. $\leftarrow$ peak.
4. back in `process_file!`: `permute_psm_by_precursor_idx!` (global sort), then (ZT only) `collapse_to_metascans` $\rightarrow$ 2.95M meta-PSMs (`MainSearch.jl:369`).

3 The locality that makes 5b correct

`collapse_to_metascans` groups the sorted table by precursor, and for each *center* scan c (where the precursor m/z lies in that scan's isolation window) it gathers the same precursor's weights at neighbor scans, binned by $d = \text{scan} - c \in [-k, k]$:

```

1 lo = max(blk_lo, r - L); hi = min(blk_hi, r + L)
2 for rr in lo:hi
3     d = Int(scns[rr]) - c
4     if -k <= d <= k
5         w[d+k+1] += wt[rr]                                # same precursor, neighbor scan
6         if _have_fragments
7             for b in 1:8; Fbuf[b][d+k+1] += fcols[b][rr]; end
8         end
9     end
10 end

```

So a meta-PSM at center c depends only on rows with scan index in $[c - k, c + k]$ of the *same precursor*. Two facts make batching byte-identical:

1. **Per-scan deconvolution is independent.** The weight of precursor p at scan s is the solution of the NNLS over the candidates *of scan s only*; it does not depend on any other scan. Hence `deconv(scan  $s$ )` yields the same rows regardless of which batch produced them — so one batch may safely *read* another batch's already-deconvolved rows.
2. **The neighbor set is a $\pm k$ scan-index window** ($k = 6$; ≤ 13 consecutive scans), tiny relative to a batch.

Deconvolve each scan exactly once. Note that consecutive centers share most of their neighbors (center c and $c+1$ overlap in $2k-1$ of their $2k+1$ scans), so batches *always* overlap in the meta-scans they touch. That overlap must *not* become duplicate deconvolution: a scan is deconvolved once, by the batch that *owns* it, and merely *read* as a neighbor by the (at most two) adjacent batches. An earlier draft of this plan deconvolved a $\pm k$ halo redundantly — rejected: it doubles work on boundary scans and adds a second site where a result could diverge.

4 Design

Gate everything on `PIONEER_ZT_METASCAN_K>0`; the conventional-DIA path is untouched.

(a) Contiguous partitioner. Replace round-robin with contiguous MS2 scan-index ranges $[s_{lo}, s_{hi}]$, one per thread, balanced by cumulative peak count. Crucially it must *not* apply the round-robin path's intra-RT-bin m/z sort (`_sort_scans_by_mz_in_rt_bins!`) — that would break scan-index contiguity.

(b) **Deconvolve once; borrow boundary rows; center ownership.** Thread t deconvolves *only* its own range $[s_{lo}, s_{hi}]$ — each scan deconvolved exactly once. For the collapse, centers in the first/last k scans of the batch need neighbors that live in the adjacent batch; those rows were *already* deconvolved by that batch, so thread t *borrow*s them (a k -scan slice filtered from the neighbor’s raw table) rather than recomputing. Emission is restricted to $c \in [s_{lo}, s_{hi}]$, so each center lands in exactly one owning batch $\Rightarrow$ no double-count, no gap, no double-deconvolution.

(c) **Collapse per batch, then vcat meta-PSMs.** Each batch’s collapse input is `head ++ rawt ++ tail` (`head/tail` = borrowed k -scan boundary slices). The `vcat` then joins ~ 3 M meta rows instead of ~ 35 M raw rows. The per-batch raw tables coexist across a barrier (their sum = today’s raw table, ~ 6.3 GB) but the 7.45 GB global `permute_by_precursor` and the `vcat`-of-raw copy are gone. A later *streaming* refinement (Tier 2) frees each `rawt` once its neighbors have collapsed, via a scan-ordered sliding window, so the raw table never fully coexists — kept separate because it serializes the (cheap) collapse in scan order.

(d) **Re-sort to global order (the subtle one).** The current global collapse emits meta rows in (`precursor_idx`, `scan_idx`) order. A batched collapse emits in (`batch`, `pid`, `scn`) order. **Light-GBM is row-order sensitive**, so the batched output must be re-sorted to (`pid`, `scn`) to reproduce the exact global order — otherwise IDs can drift even though the collapse content is identical. Re-sorting 3M rows is cheap.

5 Code changes

5.1 New: contiguous ZT partitioner

```

1 # ZT-only: contiguous MS2 scan-index ranges, balanced by cumulative peak count.
2 # Non-overlapping ownership windows -> each scan deconvolved by exactly one batch
3
4 function partition_scans_contiguous_zt(spectra, n_threads::Int)
5     ms2 = [s for s in 1:length(spectra) if getMsOrder(spectra, s) == 2] #
6         ordered
7     npk = [length(getMzArray(spectra, s)) for s in ms2]
8     total = sum(npk); target = total / n_threads
9     tasks = Vector{Tuple{Int, UnitRange{Int}}{}}() # (thread_id, own_range)
10    only
11    lo = 1; acc = 0.0
12    for i in eachindex(ms2)
13        acc += npk[i]
14        if acc >= target || i == length(ms2)
15            push!(tasks, (length(tasks)+1, lo:i)) # no halo: own range only
16            lo = i+1; acc = 0.0
17        end
18    end
19    return tasks, ms2 # ms2 maps local (ms2) index -> global scan index
20 end

```

5.2 collapse_to_metascans: add a center-ownership window

One new kwarg. When set, a center row is emitted only if its scan is owned by this batch; neighbor gathering is unchanged (it still reads halo rows present in the batch table).

```

1 function collapse_to_metascans(psms::DataFrame, spectra, precursors, k::Int;
2                               bitvec_rank_table = nothing,
3                               own_scans::Union{Nothing, Set{UInt32}} = nothing)
4                               # NEW
5     ...
6     for r in blk_lo:blk_hi
7       (abs(pm - Float32(cmzs[scn[r]])) <= Float32(hws[scn[r]]/2) || continue
8       own_scans == nothing || (scn[r] in own_scans) || continue # NEW:
9         ownership
10        c = Int(scn[r])
11        ... # unchanged
12        below
13    end
14  end
15  ...
16 end

```

own_scans is the set of global scan indices in this batch's own_range. Passing nothing recovers today's global behavior exactly (used by the non-batched path and by tests).

5.3 LibrarySearch.jl: fuse deconv + collapse per batch (ZT path)

```

1 if _zt_k > 0
2   tasks_zt, ms2map = partition_scans_contiguous_zt(spectra, Threads.nthreads())
3   metas = map(nce_entries) do (nce_model, nce_tag)
4     # --- 1. Deconvolve each contiguous range ONCE (no overlap). ---
5     rows = map(tasks_zt) do (tid, own_rng)
6       Threads.@spawn process_scans_fused!(ms2map[own_rng], spectra,
7       prec_index,
8       ms_file_idx, search_data[tid], params, precursors, ion_list,
9       nce_model, qtm_deconv, mem, rt_to_irt, irt_tol)
10    end |> ts -> fetch.(ts) # Vector of per-
11    batch raw tables
12
13    # --- 2. Collapse each batch, BORROWING neighbor rows (no re-deconv). ---
14    parts = map(enumerate(tasks_zt)) do (t, (tid, own_rng))
15      Threads.@spawn begin
16        lo_s = UInt32(ms2map[first(own_rng)]); hi_s = UInt32(ms2map[last(
17        own_rng)])
18        # k-scan boundary slices filtered from ALREADY-deconvolved
19        neighbors
20        head = t > 1 ? _rows_with_scan_ge(rows[t-1], lo_s - _zt_k) :
21        _empty_like(rows[t])
22        tail = t < length(rows) ? _rows_with_scan_le(rows[t+1], hi_s +
23        _zt_k) : _empty_like(rows[t])
24        batch_tbl = vcat(head, rows[t], tail) # borrowed rows,
25        NOT recomputed
26        own = Set{UInt32}(UInt32.(ms2map[own_rng])) # emit centers in
27        own range only
28        collapse_to_metascans(batch_tbl, spectra,
29        getPrecursors(getSpecLib(search_context)), _zt_k;
30        bitvec_rank_table = bitvec_rank_table, own_scans = own)
31      end
32    end
33    m = vcat(fetch.(parts)...) # ~3M meta rows,
34    not ~35M
35    # rows freed here (after all borrows + collapses complete)
36    nce_tag == nothing || isempty(m) || (m[!, :nce] .= nce_tag)
37  end

```

```

28         m
29     end
30     result = length(metas) == 1 ? metas[1] : vcat(metas...)
31     sort!(result, [:precursor_idx, :scan_idx])           # (d) restore
32     global order
33     # NOTE: process_file! must skip permute_psms_by_precursor_idx! and the second
34     # collapse call for the ZT path when library_search already returned meta-
35     PSMs.
36 else
37     # ... existing round-robin deconv + vcat, unchanged ...
38 end

```

5.4 MainSearch.jl: collapse already done for ZT

`process_file!` currently calls `permute_psms_by_precursor_idx!` then `process_search_results!` calls `collapse_to_metascans`. For the batched ZT path both are already handled inside `library_search`, so guard them out (a boolean on the results object, e.g. `results.already_collapsed[]`), leaving `add_chromatogram_features!` and the LGBM to run as today on the meta table.

6 Correctness / verification

1. **Boundary coverage:** assert every owned center c has its full $[c - k, c + k]$ present in the borrowed-augmented batch table (the borrowed slices guarantee it except at the file's first/last k scans, where the global run is also truncated by $\max(\text{blk_lo}, \dots) / \min(\text{blk_hi}, \dots)$).
2. **Ownership partition:** the `own_ranges` tile the MS2 scans with no overlap and no gap $\Rightarrow$ each center emitted exactly once.
3. **Content identity:** dump meta-PSMs (`PIONEER.DUMP.METASCANS`), sort both baseline and batched by `(precursor_idx, scan_idx)`, assert every column bit-identical. Baseline reference already saved: `meta_dump_baseline/meta_psms_file1.arrow`.
4. **Count invariant:** single-file collapse stays $33,185,795 \rightarrow 2,950,447$.
5. **End-to-end:** 2-file search stays $57,030 / 8,568$; a non-ZT dataset unchanged (partitioner is gated, so this should be trivially true).

7 Risks & mitigations

Risk	Mitigation
Boundary miss (dropped/short meta-PSM near a batch edge)	Borrow the k -scan slice from the neighbor’s raw table; assert full $[c-k, c+k]$ presence per owned center; the content-identity dump catches any drift.
Double deconvolution of boundary scans	Designed out: deconv partition is non-overlapping; boundary rows are <i>borrowed</i> from the neighbor’s raw table, never recomputed.
Row-order $\rightarrow$ LGBM non-determinism	Final sort! by (<code>pid</code> , <code>scn</code>) to reproduce the global order exactly.
Raw tables coexist at the deconv $\rightarrow$ collapse barrier	Tier 1 accepts ~ 6.3 GB (still drops the 7.45 GB permute + vcat copy); Tier 2 sliding window frees <code>raw_t</code> early.
Load imbalance from contiguous ranges	Balance ranges by cumulative <i>peak</i> count, not scan count.
Non-ZT path regression	Entire change gated on <code>PIONEER_ZT_METASCAN_K>0</code> .

8 Expected outcome

Peak driven down by removing the 6.34 GB raw-table materialization and the 7.45 GB global permute; the vcat shrinks from ~ 35 M to ~ 3 M rows. Target peak ≈ 22 GB $\rightarrow$ mid-teens GB (to be measured; RSS is noisy, so we compare the macOS `time -l` footprint and the alloc buckets, not a single RSS reading). Timing should be neutral-to-better (less allocation and no 7.45 GB sort), and the batches are naturally parallel.